

第2章习题

1. 简要分析 CISC 和 RISC 架构各自的优势和劣势。
2. RISC-V 中的基本指令集是什么？列举五个常见的 RISC-V 标准扩展指令集并简要说明它们的作用和应用范围。
3. RISC-V 汇编中存在许多伪指令，它们一般是具有特殊操作数的基本指令或指令组合。请写出与以下伪指令等价的基本指令或指令组合。
 - 1) nop
 - 2) ret
 - 3) call offset
 - 4) mv rd,rs
 - 5) rdcycle rd
 - 6) sext.w rd,rs
4. 阅读 RISC-V 规范以回答以下问题：
 - 1) RV32I 中的 add 指令和 RV64I 中的 addw 指令均为 32 位整型加法指令，它们是否具有相同的指令操作数（opcode）？此外，RV32I 中的 add 指令和 RV64I 中的 add 指令是否具有相同的指令操作数（opcode）？试分析为什么采取这样的设计。
 - 2) 在 RV64I 中，addw 和 addiw 指令的目标寄存器中存放的 32 位计算结果是否需要额外的符号扩展才能用于后续 64 位计算？请说明理由。
5. 什么是 RISC-V 的 I 标准指令集中存在的 HINT 指令空间？它有什么作用？
6. 考虑如下指令序列：

```
div    a2,a0,a1
rem    a3,a0,a1
```

假设寄存器 a0 和 a1 的初始值分别为 16 和 -5，则上述指令序列执行完成后 a2 和 a3 寄存器中的值分别是多少？简要说明 RISC-V 的 M 标准指令集中对除法和余数指令的符号规定。
7. RISC-V 标准指令集并未为加法指令的溢出引入专用的标志位，因此通常需要额外的指令以检查加法溢出。
 - 1) 考虑如下的指令序列：

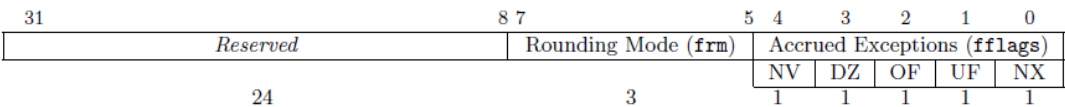
```
add t0,t1,t2
```


bne t3,t4,overflow若 t0 和 t1 都是有符号数，请在横线处填入正确的指令，使得当 t0 和 t1 的加法发生溢出时，控制流可以正确跳转到 overflow 位置。（请勿使用除 t0~t4 以外的任何寄存器）
 - 2) 当 t0 和 t1 都是无符号数时，请给出尽量简单的检测 add t0,t1,t2 指令加法是否溢出的指令序列。
 - 3) 调研其他指令集架构（如 x86、ARM 等）是如何检测加法溢出的。

8. 阅读 RISC-V 规范以了解 RISC-V 对除数为 0 的除法指令的处理方法，回答以下问题。
- 1) 对整型除法，填写下表。整型除法中除数为 0 是否会引起 RISC-V 抛出异常？试分析为什么采取这样的设计。

指令	rs1	rs2	Op=DIVU 时 rd 值	Op=REMU 时 rd 值	Op=DIV 时 rd 值	Op=REM 时 rd 值
Op rd,rs1,rs2	x	0				

- 2) 对浮点除法，除数为 0 将会引起 fcsr 控制寄存器中的相关标志位被置位。下图给出了 fcsr 的构成，请说明 fflags 的各位分别代表什么含义。fflags 被置位是否会使得处理器陷入系统调用？



- 3) 调研其他指令集架构（如 x86、ARM 等）是如何处理除数为 0 的。
9. 回答以下问题：
- 1) jal 指令包含 20 位的有符号立即数编码 (J-type)，该指令相较当前 PC 可以跳转的地址空间范围为多少？
- 2) 条件分支指令（如 bne）包含 12 位的有符号立即数编码 (B-type)，这类指令相较当前 PC 可以跳转的地址空间范围为多少？
- 3) 是否可以使用一条 lui 指令和一条 jalr 指令的组合完成任意 32 位绝对地址的跳转操作？
10. 调查 RVC 压缩指令集的编码，说明一条常用的 32 位指令能够被压缩为 16 位 RVC 指令的条件是什么？RVC 中各类型的指令是否都可以使用完整的 32 个通用整型寄存器？

11. 写出以下指令使用的寻址模式。

- 1) jal ra,0x88
- 2) jalr x0,ra,0
- 3) addi a0,a1,4
- 4) mul a0,a1,a2
- 5) ld a4,16(sp)

12. 写出以下程序在 RISC-V 中应当处于的特权等级。

- 1) Linux Kernel
- 2) BootROM
- 3) BootLoader
- 4) USB Driver
- 5) vim

13. 写出实现以下 C 程序的 32 位 RISC-V 汇编代码。假设 A 和 B 的起始地址存放于寄存器 t0 和 t1，C 的地址存放于寄存器 t2。

```
int vecMul(int *A, int *B, C){
    for(int i = 0;i < 100;++i){
        A[i] = B[i] * C;
```

```

    }
    return A[0];
}

```

14. 写出实现以下 C 程序的 32 位 RISC-V 汇编代码。假设 a、b 和 c 分别对应寄存器 a0、a1 和 a2。

```

int a, b, c;
if(a > b){
    c = a + b;
}
else{
    c = a - b;
}

```

15. 写出实现以下 C 程序的 32 位 RISC-V 汇编代码。假设指针 p 已经通过程序 `int *p = (int *) malloc(4*sizeof(int))` 得到，且 p 存放于 t0 中，a 存放于 t1 中。

```

p[0] = p;
int a = 3;
p[1] = a;
p[a] = a;

```

16. 写出实现以下 C 程序的 32 位 RISC-V 汇编代码。假设指针 a 和 b 分别存放于 t0 和 t1 中。

```

void swap(int *a, int *b) {
    int tmp = *a;
    *a = *b;
    *b = tmp;
    return;
}

```

17. 解释以下 RISC-V 汇编代码实现的功能。

```

    addi a0,x0,0
    addi a1,x0,1
    addi a2,x0,30
loop:  beq a0,a2,done
       slli a1,a1,1
       addi a0,a0,1
       j loop
done:  # exit code

```

18. 有一组操作码，它们的出现几率如下表所示。

a_i	p_i
a	0.25
b	0.20

